

Step 1 — Two Standalone MCP-Connected Specialists — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `notes_server.py` and `web_server.py` from Hint 2's Intermediate Version already exist in the same folder.

Basic Version

Approach 1 — the direct way, printing instead of returning

```
# notes_agent.py
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from openai import OpenAI

client = OpenAI()

async def main():
    server_params = StdioServerParameters(command="python", args=["notes_server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()

            openai_tools = []
            for tool in tools_result.tools:
                openai_tools.append({
                    "type": "function",
                    "function": {
                        "name": tool.name,
                        "description": tool.description,
                        "parameters": tool.inputSchema,
                    },
                })

            # Print the tools to the console
            print(openai_tools)
```

```

messages = [{"role": "user", "content": "What do my notes say about P
response = client.chat.completions.create(
    model="gpt-4o-mini", messages=messages, tools=openai_tools
)
reply = response.choices[0].message

if reply.tool_calls:
    for call in reply.tool_calls:
        import json
        args = json.loads(call.function.arguments)
        result = await session.call_tool(call.function.name, args)
        print("Tool result:", result.content[0].text)
    else:
        print(reply.content)

asyncio.run(main())

```

Expected output, if the model chooses `search_notes("Project Atlas")` :

```
Tool result: ['Project Atlas kickoff meeting notes: launch targeted for Q3.', 'Pr
```

This proves the connection, discovery, and one real tool call all work. It's a single pass, not a real loop — if the model wanted to call a second tool after seeing the first result (like reading a note by id), this version wouldn't feed the result back and ask again. That's what Intermediate adds.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a real loop, reused across both agents via a shared helper

```
# tool_bridge.py
import json

def mcp_tools_to_openai_schema(tools: list) -> list[dict]:
    """Convert MCP Tool objects into OpenAI's tools parameter shape."""
    openai_tools = []
    for tool in tools:
        openai_tools.append({
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description,
                "parameters": tool.inputSchema,
            },
        })
    return openai_tools

async def call_mcp_tool(session, name: str, arguments: dict) -> str:
    """Call an MCP tool and return its result as a plain string."""
    result = await session.call_tool(name, arguments)
    text_parts = []
    for block in result.content:
        if hasattr(block, "text"):
            text_parts.append(block.text)
    return "\n".join(text_parts)
```

```
# notes_agent.py
import asyncio
import json
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from openai import OpenAI
from tool_bridge import mcp_tools_to_openai_schema, call_mcp_tool
```

```

client = OpenAI()
SYSTEM_PROMPT = "You are a helpful assistant that answers questions using the not

async def _run_notes_agent(task: str, max_iterations: int = 5) -> str:
    server_params = StdioServerParameters(command="python", args=["notes_server.p
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            openai_tools = mcp_tools_to_openai_schema(tools_result.tools)

            messages = [
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": task},
            ]

            for _ in range(max_iterations):
                response = client.chat.completions.create(
                    model="gpt-4o-mini", messages=messages, tools=openai_tools
                )
                reply = response.choices[0].message
                messages.append(reply)

                if not reply.tool_calls:
                    return reply.content

                for call in reply.tool_calls:
                    args = json.loads(call.function.arguments)
                    result_text = await call_mcp_tool(session, call.function.name
                    messages.append({
                        "role": "tool",
                        "tool_call_id": call.id,
                        "content": result_text,
                    })

            return "Reached max iterations without a final answer."

def run_notes_agent(task: str) -> str:
    return asyncio.run(_run_notes_agent(task))

if __name__ == "__main__":
    print(run_notes_agent("What do my notes say about Project Atlas?"))

```

`web_agent.py` is the identical shape, pointed at `web_server.py` :

```

# web_agent.py
import asyncio
import json
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from openai import OpenAI
from tool_bridge import mcp_tools_to_openai_schema, call_mcp_tool

client = OpenAI()
SYSTEM_PROMPT = "You are a helpful assistant that answers questions using the fet

async def _run_web_agent(task: str, max_iterations: int = 5) -> str:
    server_params = StdioServerParameters(command="python", args=["web_server.py"]
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            openai_tools = mcp_tools_to_openai_schema(tools_result.tools)

            messages = [
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": task},
            ]

            for _ in range(max_iterations):
                response = client.chat.completions.create(
                    model="gpt-4o-mini", messages=messages, tools=openai_tools
                )
                reply = response.choices[0].message
                messages.append(reply)

                if not reply.tool_calls:
                    return reply.content

                for call in reply.tool_calls:
                    args = json.loads(call.function.arguments)
                    result_text = await call_mcp_tool(session, call.function.name
                    messages.append({
                        "role": "tool",
                        "tool_call_id": call.id,
                        "content": result_text,
                    })

            return "Reached max iterations without a final answer."

```

```
def run_web_agent(task: str) -> str:
    return asyncio.run(_run_web_agent(task))

if __name__ == "__main__":
    print(run_web_agent(
        "Fetch https://intranet.example.com/atlas/status and tell me what it says"
    ))
```

Expected output (notes agent): something like "Your notes say Project Atlas is targeted for a Q3 launch, and it's currently waiting on legal sign-off." **Expected output**

(web agent): something like "The status page says: Project Atlas status: on track, 80% complete."

Difference from Basic: Basic makes exactly one tool-call round trip and stops. Intermediate is a real loop (up to `max_iterations`), reused identically by both agents through the shared `tool_bridge.py` — this is the actual ReAct shape from Doc07 and Project 8's Step 2, and it's what lets the agent chain two tool calls together if it ever needs to (e.g. `search_notes` then a follow-up `add_note`).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a shared, reusable connection helper, and a real "no results" test

```
# mcp_connection.py
from contextlib import asynccontextmanager
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

@asynccontextmanager
async def connect_stdio_server(command: str, args: list[str]):
    """Open a fully-initialized ClientSession to one MCP server over stdio."""
    server_params = StdioServerParameters(command=command, args=args)
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            yield session
```

```
# notes_agent.py (relevant part only -- rest unchanged from Intermediate)
import asyncio
import json
from mcp_connection import connect_stdio_server
from openai import OpenAI
from tool_bridge import mcp_tools_to_openai_schema, call_mcp_tool

client = OpenAI()
SYSTEM_PROMPT = "You are a helpful assistant that answers questions using the not

async def _run_notes_agent(task: str, max_iterations: int = 5) -> str:
    async with connect_stdio_server("python", ["notes_server.py"]) as session:
        tools_result = await session.list_tools()
        openai_tools = mcp_tools_to_openai_schema(tools_result.tools)

        messages = [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": task},
        ]
```

```

    for _ in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=openai_tools
        )
        reply = response.choices[0].message
        messages.append(reply)

        if not reply.tool_calls:
            return reply.content

        for call in reply.tool_calls:
            args = json.loads(call.function.arguments)
            result_text = await call_mcp_tool(session, call.function.name, args)
            messages.append({
                "role": "tool",
                "tool_call_id": call.id,
                "content": result_text,
            })

    return "Reached max iterations without a final answer."

def run_notes_agent(task: str) -> str:
    return asyncio.run(_run_notes_agent(task))

```

```

# main.py
from notes_agent import run_notes_agent
from web_agent import run_web_agent

if __name__ == "__main__":
    print("== Notes Agent ==")
    print(run_notes_agent("What do my notes say about Project Atlas?"))

    print("\n== Notes Agent, a query with no match ==")
    print(run_notes_agent("What do my notes say about a Mars mission?"))

    print("\n== Web Agent ==")
    print(run_web_agent("Fetch https://intranet.example.com/atlas/status and summarize"))

    print("\n== Web Agent, an unknown URL ==")
    print(run_web_agent("Fetch https://intranet.example.com/does-not-exist and summarize"))

```

Expected output (abridged):

== Notes Agent ==

Your notes say Project Atlas is targeted for a Q3 launch and is waiting on legal

== Notes Agent, a query with no match ==

I couldn't find anything in your notes about a Mars mission.

== Web Agent ==

The status page reports Project Atlas is on track and 80% complete.

== Web Agent, an unknown URL ==

No content is available for that URL.

Difference from Intermediate, and what this approach adds: `mcp_connection.py` turns the nested `async with` blocks into one reusable `connect_stdio_server(...)` helper, both agents now share instead of each re-writing the same two lines. The system prompt now explicitly tells the model not to invent a note when the search comes back empty — without that line, a model will sometimes answer confidently from its own guess instead of the tool's real (empty) result, which is exactly the kind of quiet failure this step is meant to catch now, before Step 2 builds anything on top of it. `main.py` deliberately runs both agents twice each — once with a real match, once with a query that has none — so a "no results" bug is visible immediately instead of hiding behind a test that only ever asks the easy question.

Which one should you actually write? The Advanced approach's shape — a shared `mcp_connection.py`, a shared `tool_bridge.py`, and a sync wrapper (`run_notes_agent`) around the real async logic (`_run_notes_agent`) — is what you should carry into Step 2. That sync/async split matters: Step 2's graph nodes can call `run_notes_agent(task)` directly like any normal Python function, without every part of the graph needing to be `async`. Keep the "don't invent an answer when the tool found nothing" line in both agents' system prompts — it's a one-line fix that prevents a real, easy-to-miss category of bug.